

# *Számítási és tárolási felhők alapjai (6. ea)*

*Konténerizáció és konténer orkesztráció*

Szeberényi Imre, Ludmány Balázs

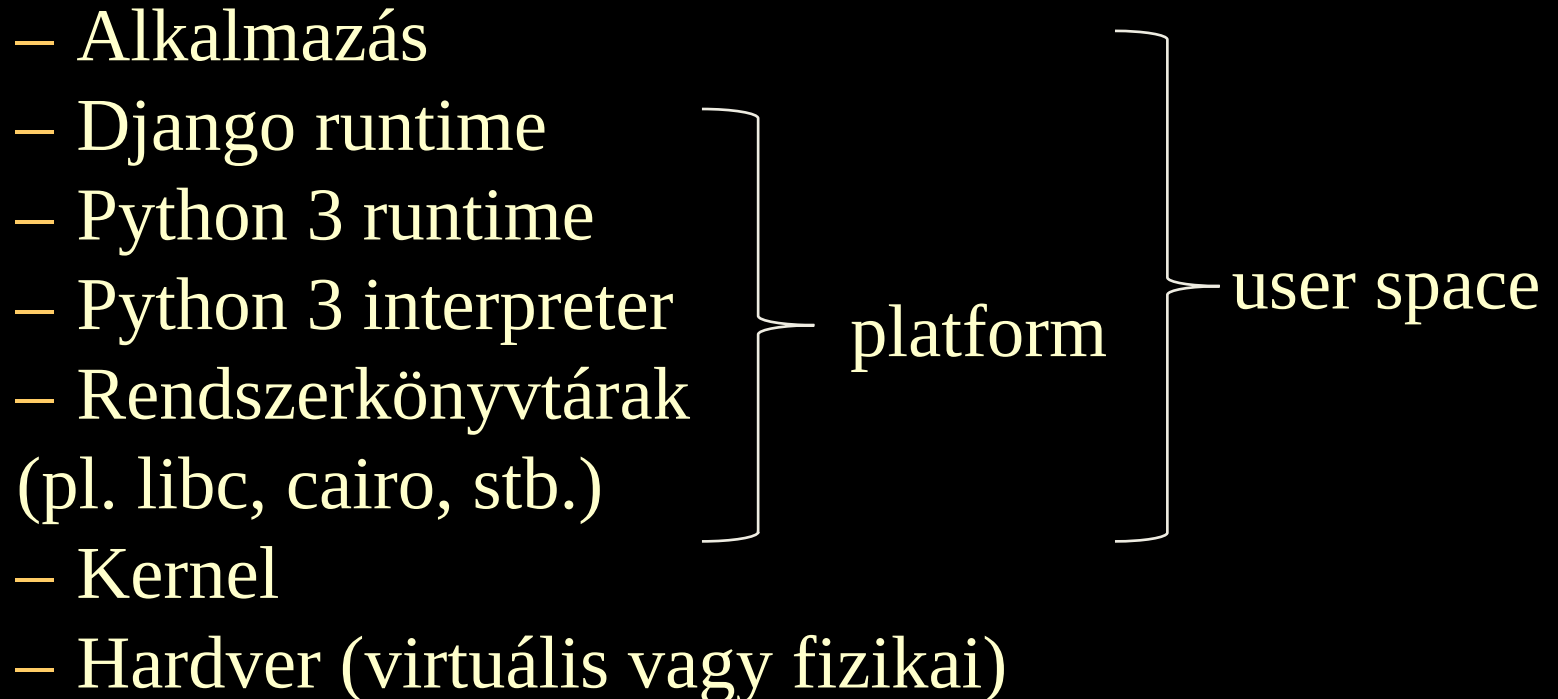
BME IIT

<szebi@iit.bme.hu>



# *Platform as a Service*

- Platform: a saját szoftverünk összes függősége, ami a futtatásához szükséges
- Például Django alkalmazás:



# *Op. rendszer szintű virtualizáció*

## **CPU virtualizáció**

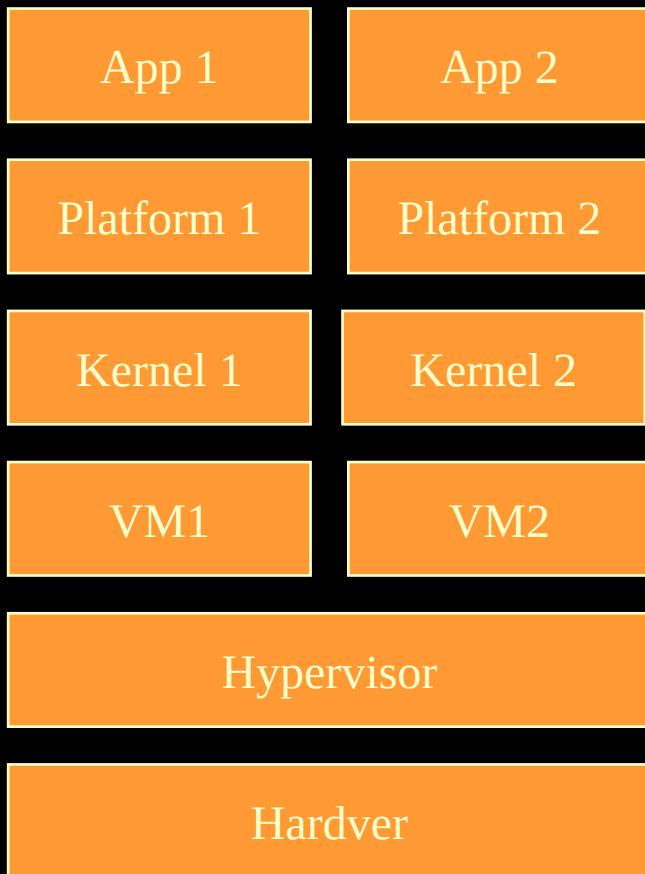
- Több izolált virtuális gép ugyanazon a fizikai gépen
- Hardverkomponensek emulációja
- Hypervisor irányítja

## **OS szintű virtualizáció**

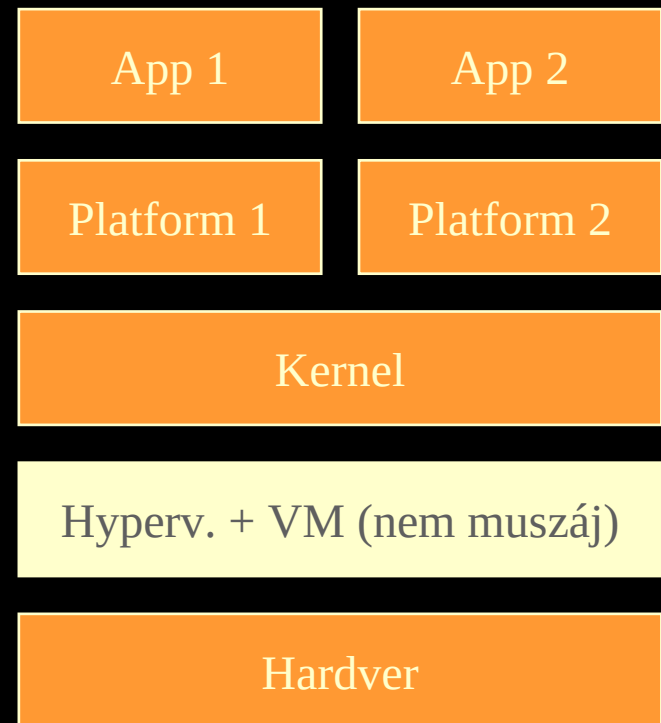
- Több izolált user space ugyanazon a kernelen
- Fájrendszer, felhasználói fiókok és csoportok, processz fa, IPC stb. emulációja
- A kernel nyújt hozzá támogatást

# *Op. rendszer szintű virtualizáció*

## CPU virtualizáció



## OS szintű virtualizáció



# *Op. rendszer szintű virtualizáció*

## **CPU virtualizáció**

- Tetszőleges szoftver telepíthető a virtuális gépre
  - verzió
  - konfiguráció
- Jobb szeparáció a többi bérlőtől multitenant környezetben

## **OS szintű virtualizáció**

- Kisebb overhead
- Gyorsabb indulás

# *Linux kernel támogatás #1*

- **chroot (change root):** parancs Unix és Unix-szerű rendszereken, ami megváltoztatja, hogy egy processz (és gyerekei) melyik mappát látják root-ként
- **cgroups (control groups):** processzek egy csoportjának erőforráshasználatát tartja számon, és korlátozza
  - o CPU cgroup
  - o Memory cgroup
  - o Block I/O cgroup
  - o Device cgroup
- Az OS szintű virtualizáció támogatása egyéb komponensekben (pl. SELinux, iptables).

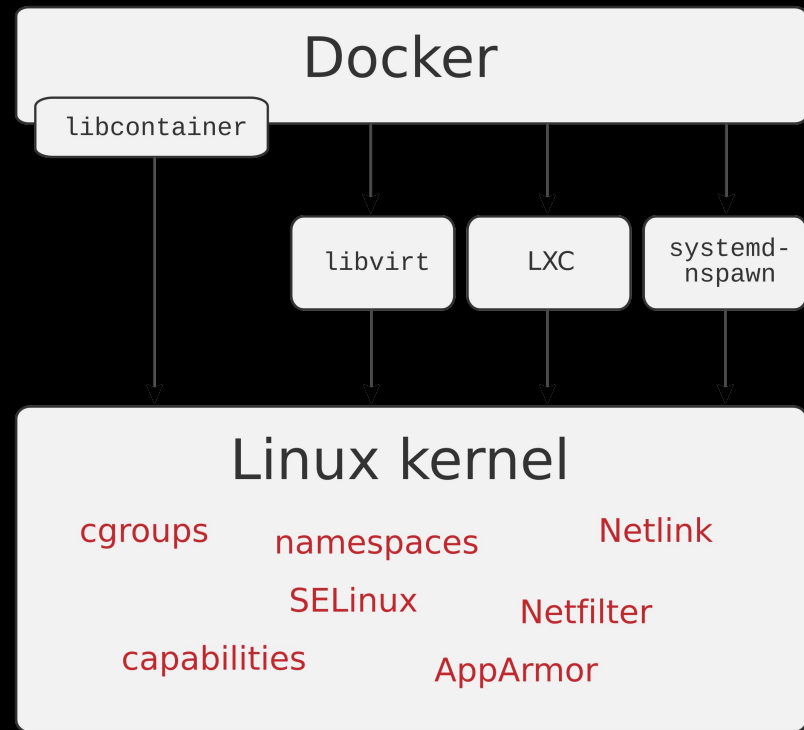
# *Linux kernel támogatás #2*

– **namespaces:** processzek csoportosítását teszi lehetővé, más névtérbe tartozó processzek más erőforrásokhoz férhetnek hozzá

- User namespace: izolált UID/GID-ek a konténeren belül, amiket leképez a hoszt UID/GID-jeire
  - Pl.: 0-s (root) UID a konténerben, de kívül nem
- Process ID namespace
- Network namespace
- Mount namespace
- IPC namespace
- UTS namespace: konténerenként eltérő hoszt és domain név
- Cgroup namespace

# Docker

- 2011: dotCloud Inc.
- 2013: Docker Inc.
- A szoftvereket *konténerekbe* csomagoljuk
- A konténereket az OS szintű virtualizáció segítségével futtatjuk
- 0.9-ig LXC-re épül
- 0.9-től a saját libcontainer könyvtárakra



# *Docker terminológia #1*

- **Container:** szabványos izolált környezet ami tartalmazza az alkalmazásunkat és a platformot, amin fut
- **Image:** írásvédett sablon, ebből kiindulva építjük fel a konténereinket
- **Registry:** image-ek nyilvántartása

```
docker pull python
```

- **Engine:** a konténereket ténylegesen futtató komponens
- **Daemon:** menedzseli a konténereket

# *Docker terminológia #2*

- **Dockerfile:** a szoftverünk forráskódjának része, amely megadja, hogy egy image-ből hogyan álljon elő a konténer

```
FROM ubuntu:latest
COPY ./examplefile.txt /examplefile.txt
ENV MY_ENV_VARIABLE="example_value"
RUN apt-get update
```

```
# Mount a directory from the Docker volume
# Note: This is usually specified in the
# 'docker run' command.
VOLUME ["/myvolume"]
```

```
# Expose a port (22 for SSH)
EXPOSE 22
```

# *Docker terminológia #3*

- **CLI:** kliens program a konténerek menedzselésére

```
docker build # konténer létrehozása az image-ből  
              # a Dockerfile utasításai szerint  
docker run   # a konténer futtatása
```

- A run parancs paraméterlistájában felülírhatjuk a Dockerfile egyes utasításait, személyre szabhatjuk a konténert
  - elérhetővé tehetjük a hoszt egyes mappáit a konténerből
  - portokat nyithatunk, amin a konténer kommunikál
  - hardvert (pl. GPU) tehetünk elérhetővé a konténerből

# *Konténerizáció — példák*

- **Docker**
- **OpenVZ**
- **LXC** (Linux Containers)
- **FreeBSD jails**
- **Podman**
- **Apptainer** (korábban Singularity, HPC környezetben)
- **Sarus** (HPC környezetben)

# *Több konténer, klaszteren*

- Docker Compose
  - o Multi-konténer alkalmazások
  - o `docker-compose.yml`: az alkalmazás minden konténerének konfigurációja
  - o `docker compose`: az alkalmazás futtatása
- Docker Swarm
  - o Klaszter menedzser
  - o Különböző gépeken futó Docker Engineket egy virtuális Docker Enginként fog össze
- A két technológia együtt is használható

# *Külső klaszter menedzserek*

- **Apache Mesos**
- **Proxmox**
- **Kubernetes**
  - 2014, Google, a **Borg** belső projekt utódjaként
  - Az 1.0-s verzióval együtt létrejött a Cloud Native Computing Foundation (CNCF) a Linux Foundation-on belül
  - Cloud native computing: skálázódó alkalmazások fejlesztése modern, dinamikus környezetekben mint a felhő
- **OpenShift**
- **Amazon Elastic Container Service (ECS)**

# Kubernetes terminológia #1

- Pod:
  - o Tipikusan egyetlen konténer életciklusáért felelős, de lehet benne több is, amik egyszerre indulnak és állnak le
  - o A legkisebb futtatható egység a Kubernetesen

Pending      A pod az indítására vár, pl. még töltődik le a konténer image.

Running      Minden konténer létrehozva, és legalább az egyik fut vagy éppen (újra)indul.

Succeeded    Minden konténer sikeresen leállt, és nem indulnak újra.

Failed        Minden konténer leállt, legalább egy hibával.

Unknown      A pod állapota nem állapítható meg (pl. kommunikációs hiba)

- o Minden pod önálló belső IP címmel rendelkezik, ezen osztoznak a benne futó konténerek
- Node: podokat futtató (virtuális vagy fizikai) gép

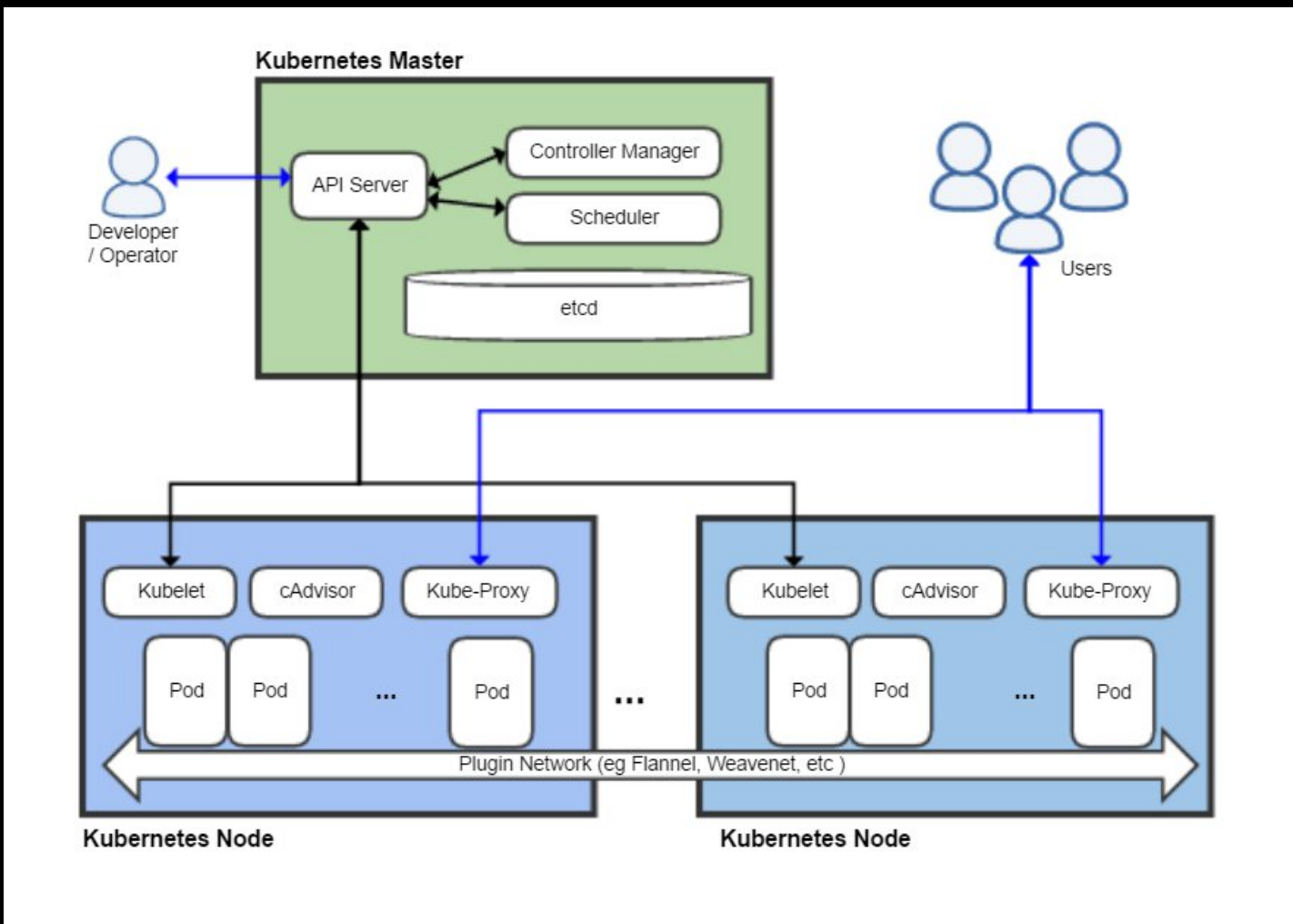
# *Kubernetes terminológia #2*

- Replica set
  - A horizontális skálázás legalsó szintje
  - Meghatározott számú *ugyanolyan, állapotmentes* podot futtat
- Deployment
  - Lehetővé teszi a podok fokozatos frissítését a replica seten belül
  - Hiba esetén rollback
- Stateful set
  - Meghatározott számú *ugyanolyan, állapottal rendelkező* podot futtat
  - Pl. adatbázisok, üzenet brókerek futtatására
- Daemon set
  - Node-onként egy példányban fut
  - Pl. monitorozó ágensek futtatására

# *Kubernetes terminológia #3*

- Service
  - Stabil nyilvános IP címet vagy hosztnevet rendel podok egy csoportjához
  - A szolgáltatást alkotó podok belső IP címe még változhat!

# Vezérlő sík (control plane)



# *Erőforrás kérés és határ*

- Informálisan egy pod *méretének* hívjuk az erőforrás felhasználását (nagy pod > kis pod)
- RAM: bájt, kilobájt, megabájt stb.
- 1 CPU = 1 mag 100%-os kihasználása
  - 2 CPU = 2 mag 100%-os kihasználása
  - 200 milliCPU = 1 mag 20%-os kihasználása
- API-n keresztül tetszőleges egyéb erőforrás
- Erőforrás kérés (resource request)
  - Megadja, hogy a pod egy példánya várhatóan mennyit fog az adott erőforrásból használni
  - Ez alapján dönti el a Kubernetes ütemezője, hogy melyik node-on lesz a leoptimálisabb elindítani a podot
- Erőforrás határ (resource limit)
  - Ennél többet nem enged használni a podnak

# Automatikus skálázás

- A használt számítási erőforrásokat dinamikusan a terheléshez igazítjuk

Mindig álljon rendelkezésre az optimális felhasználói élményhez szükséges kapacitás

VS

Ne fizessünk a szükségesnél több erőforrásért

- Fel- és leskálázás
- Horizontális és vertikális skálázás

# *Skálázás szintjei Kubernetesben*

## **Alkalmazásszintű skálázás**

- Horizontal Pod Autoscaler (HPA): azonos méretű podok számának változtatása a felhasználói terhelés függvényében
- Vertical Pod Autoscaler (VPA): fix számú pod méreteinek változtatása a felhasználói terhelés függvényében

## **Klaszterszintű skálázás**

- A node-ok változtatása a futtatandó podok függvényében
- Horizontális: plusz node-ok indítása vagy leállítása
- Vertikális: kisebb/nagyobb node indítása, podok migrálása
- Cluster Autoscaler, Karpenter
- Alatta pl. AWS Auto Scaling group

# *Horizontal Pod Autoscaler #1*

- Kubernetes controllerként fut
- Tetszőleges deployment vagy stateful set horizontális skálázására beállítható, de nem muszáj az összesre beállítani!
- Mindig annyi podot futtat, hogy egy vagy több metrika minél közelebb legyen egy általunk konfigurált célértékhez.
- Példák metrikákra:
  - Kihasználtság (utilization): egy pod egy adott erőforrásra vonatkozó resource requestjének hány százalékát használja
  - Másodpercenként kiszolgált kérések
  - Üzenetsor hossza: ha túl sok üzenet vár feldolgozásra, akkor újabb példányt indít

# Horizontal Pod Autoscaler #2

Egy deployment vagy stateful set skálázásának a lépései

1. Szabályos intervallumonként (alapértelmezetten 15 sec) lekéri a benne lévő podok figyelt metrikáit
2. Átlagolja az egyiket a podokra, és veszi ennek arányát a célértékkel
3. Kiszámítja a replikák optimális számát:

$$desiredReplicas = ceil \left| currentReplicas \times \frac{currentMetricValue}{desiredMetricValue} \right|$$

4. Megismétli a számítást minden metrikára, és veszi a maximumát
5. Frissíti a deploymentet vagy stateful setet

# *Horizontal Pod Autoscaler #3*

- **Min/max korlát:** megadhatunk olyan minimum/maximum replika számot, ami alá/fölé semmiképp nem megy
- **Tolerancia:** ennél kisebb eltérést a célérték és a metrika aktuális értéke közt már egyenlőségnek vesz
- **Stabilizációs ablak:** a túl nagy fluktuáció elkerülésére nem csak egy adott pillanatban, hanem egy csúszóablakban számítja a replika számot, és ezek maximumát veszi
- **Scaling policy:** bonyolultabb skálázási algoritmus meghatározására

# *Számítási és tárolási felhők alapjai (10. ea)*

*Tárolók, hálózati technológiák,  
referencia modell, üzenetsorok*

Ludmány Balázs, Szeberényi Imre  
BME IIT

<szebi@iit.bme.hu>



MŰEGYETEM 1782

# *Tároló eszközök*

- Az adatközpontok speciális tárolórendszerekkel rendelkeznek, amelyek számos merevlemezt tartalmaznak tömbökbe szervezve.
- A merevlemez-tömbök elosztják és replikálják az adatokat több fizikai lemez között. Tartalék lemezek beépítésével növelik a teljesítményt és a redundanciát.
- Redundant Arrays of Independent Disks (RAID) sémák különböző szintű adatbiztonságot nyújtanak.

# *Tároló eszközök #2*

- I/O Caching – gyorsítótárazás javítja a lemezelérési időt és teljesítményt
- Hot-Swappable Hard Disks – üzem közben cserélhető lemezegység
- Storage Virtualization – virtualizált merevlemezek és tárhelymegosztás révén valósul meg
- Fast Data Replication – Snapshot, klónozás memóriába másolás

# Tároló eszközök #3

- Robotizált tárolók, szalagos könyvtárak (tape library), melyek legtöbbször biztonsági mentésre szolgálnak.
- Ezek elérhetők hálózati informatikai erőforrásként (NAS, SAN), vagy közvetlen csatolású tárolóként (DAS), amelyben egy tárolórendszer közvetlenül kapcsolódik a számítástechnikai erőforráshoz.
- NAS, SAN – elérésnél a rendszert kiegészíti egy diszk alrendszer is, ami a használatot egyszerűsíti.



# RAID

- Redundant Array of Independent Disks



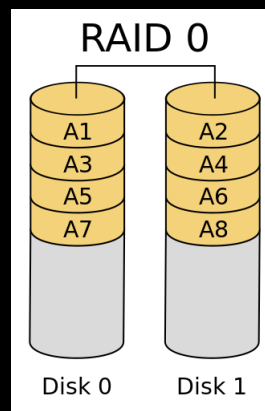
- RAID levels:
  - Level of the redundancy and protection scheme
  - Some part of the storage may store parity or error correction code.

<https://en.wikipedia.org/wiki/RAID>

# RAID #2

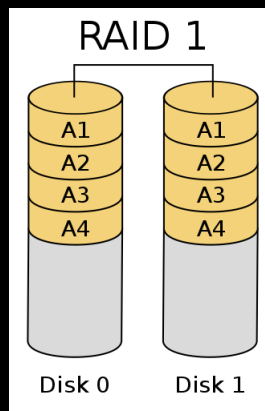
- RAID 0

- no redundancy
- only striping
- efficiency = 1



- RAID 1:

- mirroring
- efficiency = 1/2

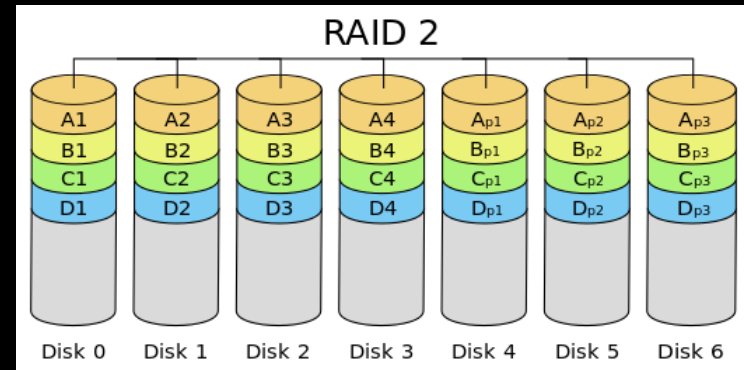


# RAID #3

- RAID 2

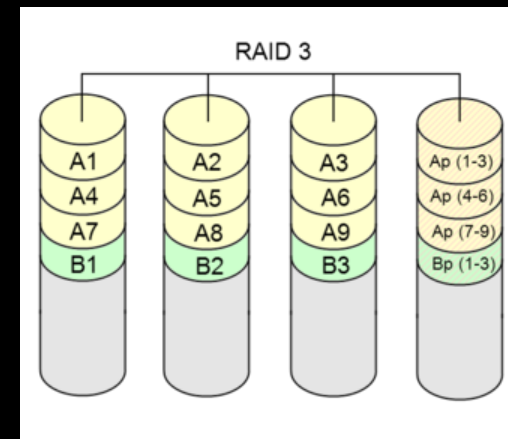
- ECC (Hamming)
- error detection
- error correction

- $\text{eff} = 2^m - m - 1, m = \log_2(n + 1)$



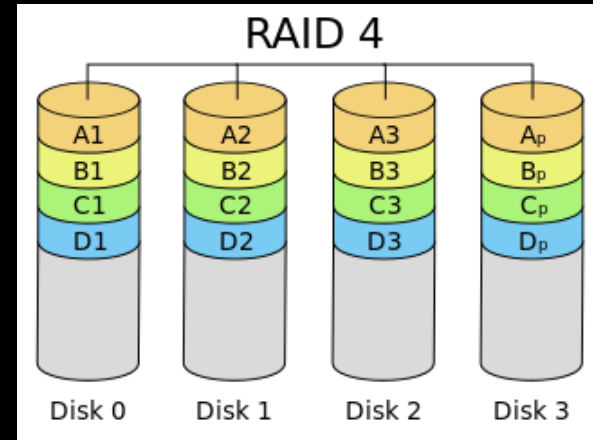
- RAID 3:

- XOR
- small stripes
- single „user” mode
- $\text{eff} = 1 - \frac{1}{n}$

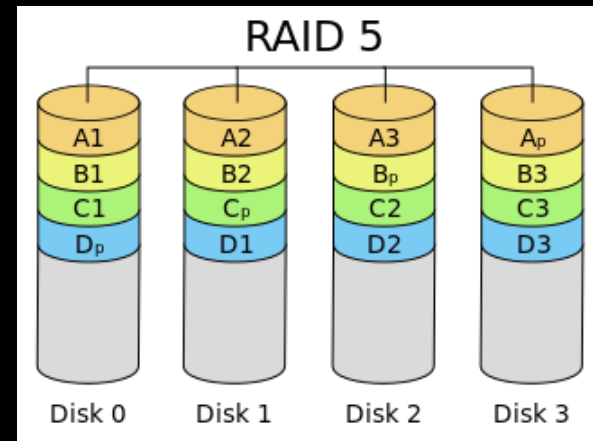


# RAID #4

- RAID 4
  - same as RAID 3
  - block level stripes
  - multi „user”
  - $\text{eff} = 1 - \frac{1}{n}$



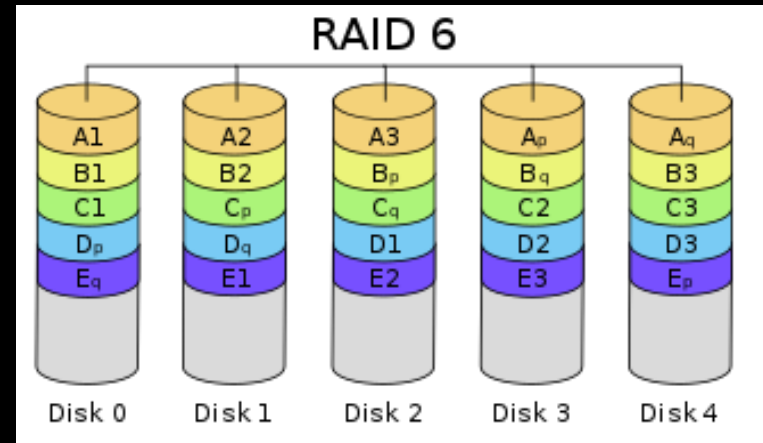
- RAID 5:
  - same as RAID4
  - rotating parity
  - $\text{eff} = 1 - \frac{1}{n}$



# RAID #5

- RAID 6

- same as RAID 5
- + column parity
- $\text{eff} = 1 - \frac{2}{n}$

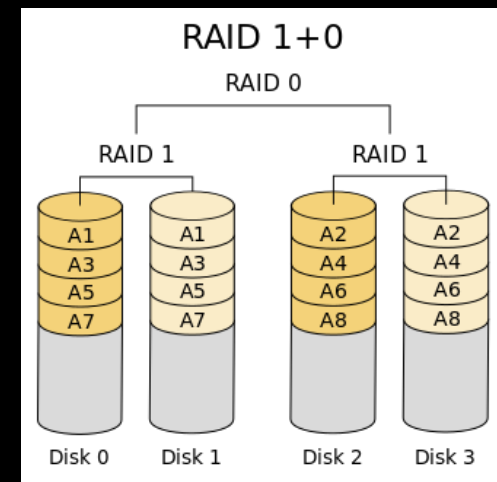
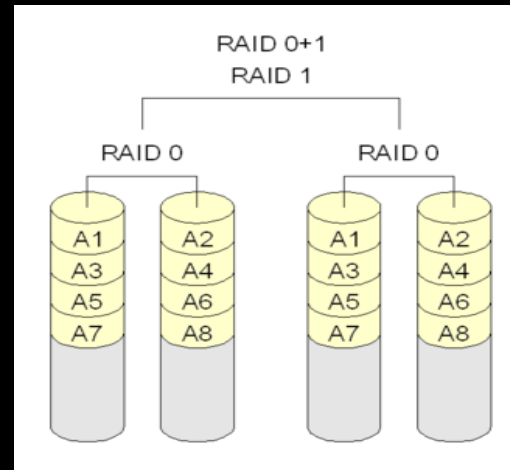


- RAID 7

- special real-time os + processors for enhanced r/w (raid level 3 or 4)

# RAID #6

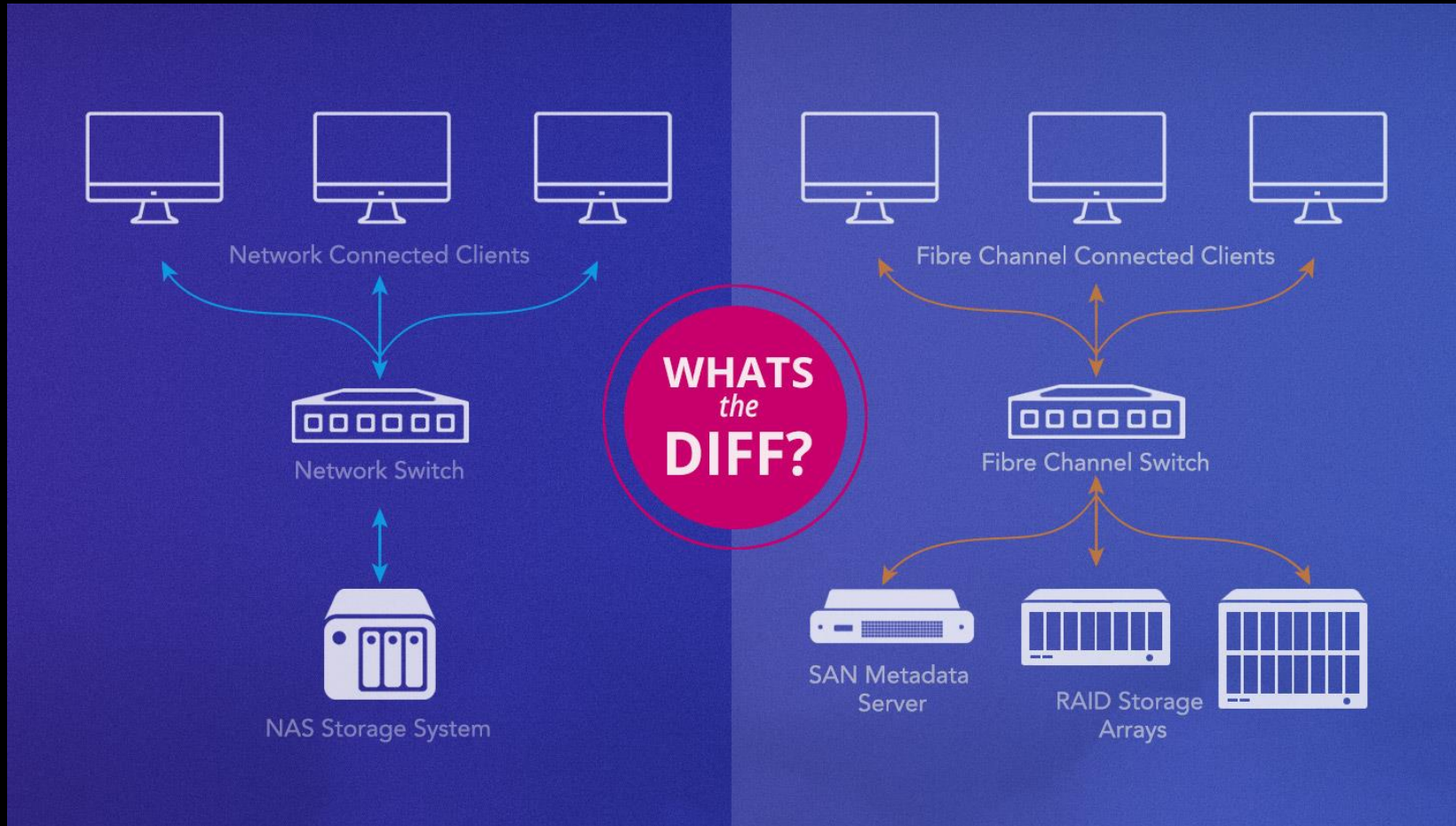
- RAID 01:
  - RAID 0 + 1
  - better speed
  - efficiency =  $2/n$
- RAID 10:
  - RAID 1 + 0
  - better protection because faulty disk effects only the mirror
  - efficiency =  $2/n$



# SAN, NAS

- Storage Area Network (SAN) – Az adattároló eszközökhöz blokk szintű hozzáférést biztosít egy dedikált hálózaton keresztül szabványos protokollal pl. Small Computer System Interface (SCSI)
- Network-Attached Storage (NAS) – A merevlemezeket tartalmazó eszköz hálózaton keresztül csatlakozik, amely fájl szintű elérést biztosít pl. Network File System (NFS) vagy a Server Message Block (SMB) protokollal.

# *SAN, NAS #2*



# *Hálózati technológiák*

Az adatközpontok hálózata öt hálózati alrendszerre osztható:

- **Carrier and External Networks Interconnection:**
  - Általában a gerinchálózati útválasztókra értik, amelyek a külső WAN-kapcsolatok és az adatközpont LAN-hálózata között biztosítanak útválasztást, valamint a peremhálózati biztonsági eszközöket, például a tűzfalakat és a VPN-átjárókat.
- **Web-Tier Load Balancing and Acceleration**
  - Webgyorsító eszközök, például XML előfeldolgozók, titkosító/visszafejtő készülékek és 7. rétegű kapcsolóeszközök, amelyek tartalomfüggő útválasztást hajtanak végre.

# *Hálózati technológiák #2*

- LAN Fabric
  - Belső LAN-t, és redundáns csatlakozást biztosít az adatközpont összes hálózati informatikai erőforrásának.
  - A hálózati kapcsolók számos virtualizációs funkciót is elláthatnak, mint például a LAN-ok VLAN-ok használatát, link-aggregációt, terheléselosztást.
- SAN Fabric
  - Tárolóhálózatok (SAN) megvalósítása.
  - SAN-t általában Fibre Channel (FC), Fibre Channel over Ethernet (FCoE) és InfiniBand hálózati kapcsolókkal valósítják meg.

# *Hálózati technológiák #3*

- NAS Gateways
  - Átjárók a NAS-alapú tárolóeszközökhöz, és olyan protokollkonverziós hardvert tartalmaz, amely megkönnyíti a SAN és NAS-eszközök közötti adatátvitelt.

A fenti öt hálózati alrendszer emeli az adatközpontok redundanciáját és megbízhatóságát, hogy elegendő informatikai erőforrással rendelkezzenek a szolgáltatások megfelelő szintjének fenntartásához még többszöri meghibásodás esetén is.

# *Adatközpontok — összefoglalás*

- IT erőforrások közös táplálással, kommunikációs vonalakkal, felügyelettel.
- Automatizálás, távoli menedzsment
- IaaS (PaaS, SaaS) szolgáltatások
- CPU virtualizáció
  - A hypervisor feladata
  - Full és paravirtualizáció
  - Megismert technológiák: KVM és libvirt
  - Memóriagazdálkodás és migráció

# *Adatközpontok — összefoglalás*

- Tároló eszközök
  - RAID
  - SAN, NAS
- Megosztott és elosztott fájlrendszerek
  - AFS: kliens és szerver
  - Lustre, Ceph: kliens, szerver és metaadattár
  - ZFS: storage pool, COW, checksum
- Hálózati technológiák

# Üzenetsorok

- Egyszerű pont-pont adattovábbítás
- Feladatsorok – üzenetek feladatokat definiálnak a worker(eknek)
- Publish/Subscribe sorok
  - Üzenetek, Témák, Előfizetők, Publikálók
- Routolható üzenetsorok
- RPC (request/reply-pattern)

# RPC (Remote Procedure Call)

- A „Remote Procedure Call” programozási modell a **request/reply mintára épül**:  
a kliens egy *request* üzenetet küld a szervernek, és egy *reply* üzenetben kap választ.

| Szint               | Fogalom                            | Leírás                                       |
|---------------------|------------------------------------|----------------------------------------------|
| Alkalmazási szint   | <b>Remote Procedure Call (RPC)</b> | Távoli függvényhívás logikai modellje        |
| Üzenetküldési szint | <b>Request / Reply Pattern</b>     | Az RPC üzenetváltásának megvalósítása        |
| Implementáció       | <b>Celery / RabbitMQ / gRPC</b>    | Konkrét rendszer, ami ezt a mintát használja |

# RPC történelem

| Időszak      | Technológia                                            | Lényege                                    | Fő gondolat                                       |
|--------------|--------------------------------------------------------|--------------------------------------------|---------------------------------------------------|
| <b>1980s</b> | <b>ONC RPC<br/>(SunRPC)</b>                            | C alapú, hálózati<br>függvényhívás         | Alap RPC-minta<br>(stub + skeleton)               |
| <b>1990s</b> | <b>DCE RPC,<br/>CORBA, DCOM</b>                        | Objektum-szintű<br>RPC (IDL, ORB)          | Platformfüggetlen<br>objektumhívás, de<br>komplex |
| <b>2000s</b> | <b>SOAP / XML-<br/>RPC / .NET<br/>Remoting</b>         | HTTP feletti RPC                           | Túl bonyolult,<br>XML nehézkes                    |
| <b>2010s</b> | <b>REST, gRPC</b>                                      | Laza csatolás,<br>JSON/HTTP2,<br>streaming | Egyszerűség,<br>interoperabilitás                 |
| <b>2020s</b> | <b>Celery,<br/>RabbitMQ,<br/>Kafka RPC<br/>pattern</b> | Üzenetsoros<br>aszinkron RPC               | Skálázható,<br>hibatűrő, felhőre<br>szabott       |

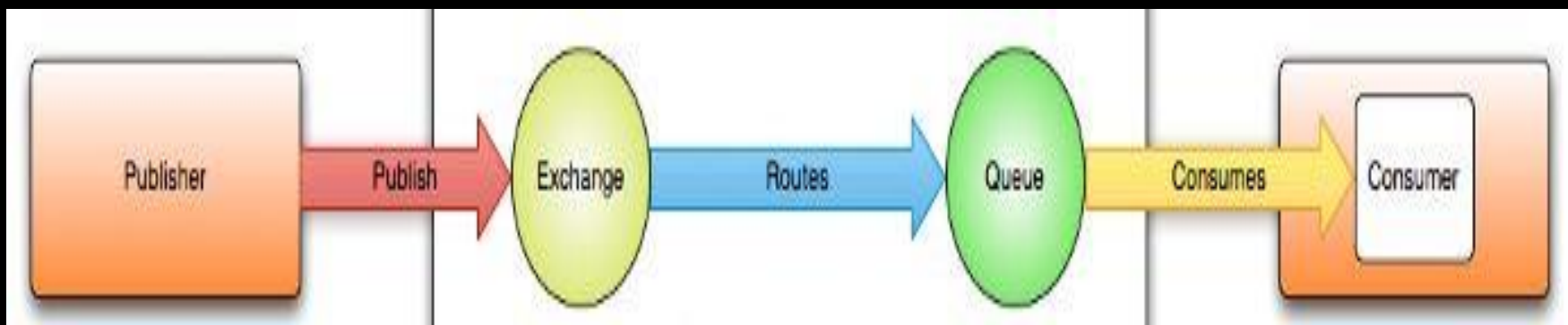
# RabbitMQ

- AMQP szabvány megvalósítása
  - Erlang
- Nagyon sok mai nyelvhez API
- Aszinkron üzenetek + RPC
- Számos routing lehetőséget támogat
  - Direkt
  - Fanout
  - Topic
  - Headers

A Celery valójában nem más, mint az RPC új ruhában – csak most a hálózati drót helyett egy nyúl (RabbitMQ) hordja a hívásokat.

# AMQP modell

- Exchange osztja el az üzeneteket az üzenet attribútumai alapján, ill. az exchange típusa szerint (direkt, fanout, ...)
- Exchange és a queue is lehet többek között durable/auto-elete, exclusive



# *AMQP fogalmak*



- Exchanges
- Queues
- Bindings
- Consumers (push, pull)
- Publishers
- Methodes
- Connetions
- Channels
- Virtual Hosts

# *Mik a feladatsorok (Task Queues)?*

---

- Taszkok sora
- Üzenetek sora, melyek mindegyike egy-egy feladatot, vagy feladat részt határoz meg.
- Az üzeneteket egy broker osztja szét
- A feladatok aszinkron módon végrehajthatók.

# *Mikor használjuk?*

- Feladat aszinkron módon hatható/hajtandó végre.
- Elosztott rendszerekben valahol máshol.
- Architektúrális megfontolásból - aszimmetria
- Processz bázisú párhuzamosítás
  - Python: GIL probléma megkerülése
    - Taszkok önállóak, nem kell lock
  - Message passing vs. shared memory párh.

# *Példák a használatra*

- E-mail küldése, miután regisztrált valaki
- Képek előfeldolgozása
- Generált, komplex adatok frissítése
- Riport készítés
- Data-paralell feladatok

# Celery

- „Szabványos” Python modul
- Egyszerű, de komplex feladatokra is jó
  - Számos üzenetsort támogat
  - Komplex folyamatokat is létre lehet hozni vele
  - Számos routolási lehetőséget támogat

```
1 from celery import Celery
2
3 app = Celery()
4
5 @app.task
6 def add(x, y):
7     return x + y
```

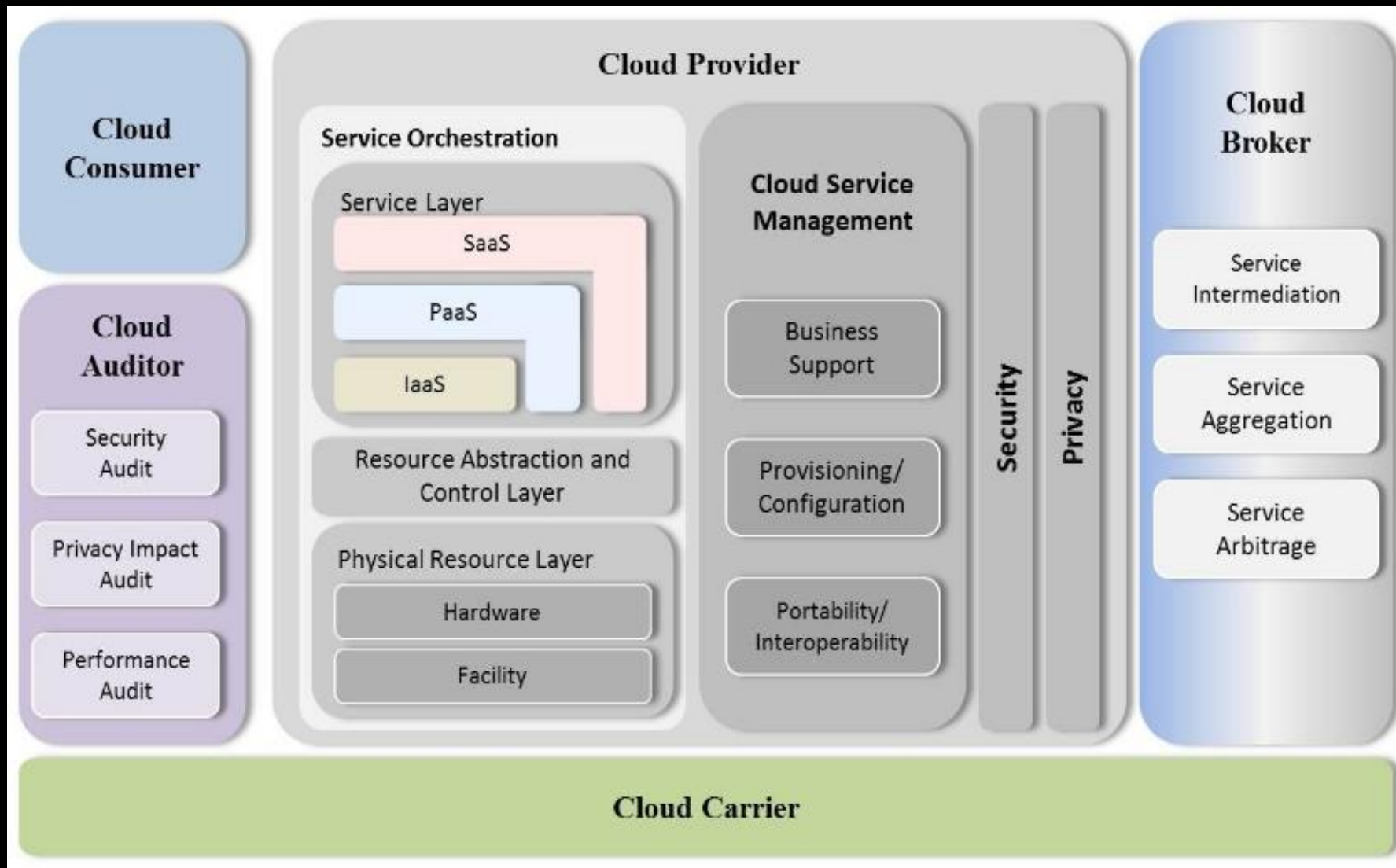
```
1 def client():
2     res = add.delay(1, 2)
3     return res.get()
4
5 def worker():
6     app.worker_main()
```

# *Más könyvtárak*

---

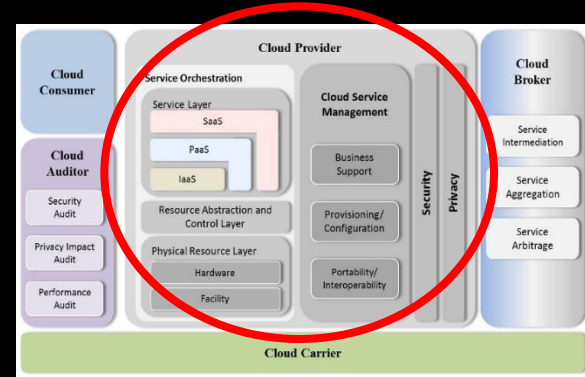
- RQ (Redis Queue)
  - Hasonló, de nagyon egyszerű
- Huey
  - Celery egyszerűbben
- Dramatiq
  - Celery, de rosszul dokumentált
- TskTiger
  - Nincs visszatérő info
- Dask – elosztott parallel ütemező

# Referencia modell (NIST)



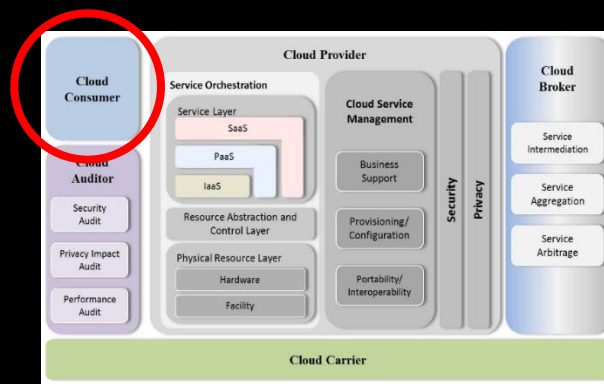
# *Felhő szolgáltató (provider)*

- Az a szervezet, ami a felhő alapú IT erőforrásokat szolgáltatja.
- A felhasználókkal (consumer) szerződésben garantálja (vagy nem) a szolgáltatási szintet (SLA).
- A működtetésért felelős.



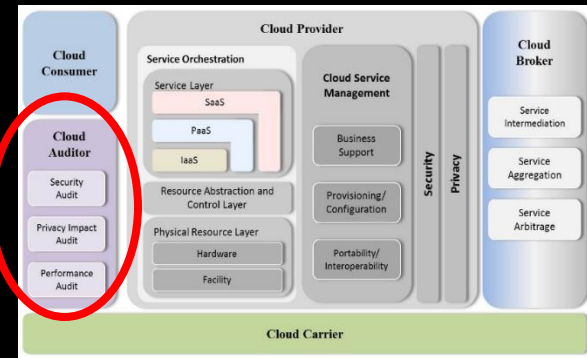
# *Felhő felhasználó (consumer)*

- Az a szervezet, vagy személy ami/aki a felhő alapú IT erőforrásokat használja.
- A szolgáltatóval (provider) szerződést/ megállapodást köt.
- A szerződés rá vonatkozó pontjainak betartásáért felelős.



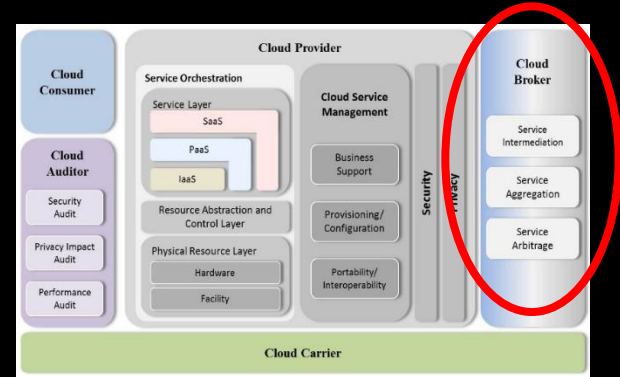
# Felhő auditor

- Független szervezet, aki biztonsági, adatvédelmi, és teljesítmény auditokat végez.
- Erősíti a bizalmat a szolgáltató és a felhasználó között.



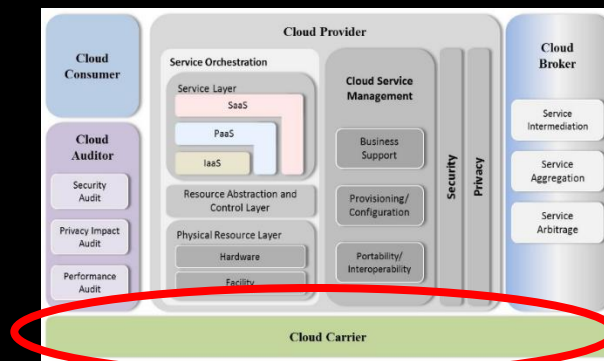
# Felhő bróker

- A felhasználó és a szolgáltató közé ékelődve elosztó, szétosztó, szerepe van.
- Igény szerint képes erőforrások egyesítésére és elosztására.



# Felhő szállító

- A felhasználó és a szolgáltató közötti összeköttetést szolgáltatja.
- Legtöbbször ez a hálózatot szolgáltató telekommunikációs cég.



# *Optionális gyakorlat*

Csatlakozás egy bugyuta kő-papír-olló  
szerverhez rabbitmq-val és celery-vel

# *Laborfeladatok Celery-vel*

---

Főbb lépések:

- rabbitmq-szerver install
- virtualis python környezet létrehozása
- mintapélda letöltése és kipróbálása
- Kő/papír/olló kliens készítése az egyik gépen futó szerverhez.

# Feladatok 1-3

1. Indítson egy gépet a `fured.cloud.bme.hu` felhőben a **Cloud Computing – libvirt** vagy a **Cloud Computing – libvirt + xfce** sablonból! (utóbbi grafikus felületű)
2. Lépjen be és frissítse a gépet:  
> `sudo apt-get update`
3. Installálja a `rabbitmq-server`-t:  
> `sudo apt-get install rabbitmq-server`

# Feladatok 4-6

4. Ellenőrizze, hogy rendben elindult-e:  
> `systemctl status rabbitmq-server`
5. Engedélyezze a menedzsment plugint  
> `sudo rabbitmq-plugins enable  
rabbitmq_management`
6. Indítson egy böngészőt a localhost:15672 URL-re, ahol meg kell jelennie a management konzolnak. User:`guest/guest`

# Remote elérés

Amennyiben nem a lokális hoszton fut a browser, úgy biztonsági okokból fel kell venni egy új usert, akinek admin jogot kell adni.

```
> sudo rabbitmqctl add_user admin NagyonTitkosJelszo  
> sudo rabbitmqctl set_user_tags admin administrator  
> sudo rabbitmqctl set_permissions -p / admin ".*" ".*" ".*"
```

Majd ki kel nyitni a tűzfalat a lokális gépen:

```
> sudo ufw allow 15672/tcp
```

Végül circle cloud dashbordján is ki kell nyitni a 15672-es portot, amiket kapunk egy portforwadot.

# Feladatok 7-8

7. Hozzon létre egy *ctest* virtuális Python környezetet és könyvtárat:

> *mkdir ctest*

> *mkvirtualenv ctest -p /usr/bin/python3*

8. Lépjen be a virtuális környezetbe és installálja ebben a Celery modult:

> *workon ctest; cd ctest*

> *pip install celery*

# *Több terminálablak*

A worker és a kliens futtatásához több terminálablakra lesz szükségünk. Ehhez alfanumerikus környezetben indítsunk most egy *screen*-t. Screen gyorstalpaló:

*ctrl-a ?* – help

*ctrl-a c* – új ablak

*ctrl-a n* – következő ablak

*ctrl-a |* – függőleges ablakosztás

*ctrl-a S* – vízszintes ablakosztás

*ctrl-TAB* – következő fókusz

*ctrl-k* – ablak bezárása

*ctrl-q* – osztás megszüntetése (Q – mindent megszüntet)

# Feladatok 9-10

9. Töltse le a mintapéldát:

> *git clone https://git.iit.bme.hu/felho2/celery.git .*

10. A ctest.py file az egyszerűség kedvéért tartalmazza a workert és a klienst is: modulként használva worker, szkriptként pedig kliens. Indítsa el a workert:  
> *celery -A ctest worker*

# Feladatok 11

11. Nyisson egy másik terminálablakot és abban indítsa el a klienst:

> *workon ctest; cd ctest*

> *python ctest.py*

Fontos, hogy ebben az ablakban is azonos python virt. környezetben dolgozzon! Ha új ablakot nyitott a grafikus felületen, vagy újból belépett, akkor kell a

> *workon ctest; cd ctest*

# *Feladatok 12*

## 12. Elemezze a mintafeladat működését!

Az `add.s(...)` jelölés egy még nem indított, de paraméterezett task-objektumot hoz létre, ami láncolást tesz lehetővé.

Tetszése szerint módosítsa a kódot!

A kommentelt 5. és 6. sor a Celery másik fajta inicializálását/paraméterezését mutatja. Próbálja ki! Törölje a kommentet!

# Feladat 13

13. Készítsen egy kliens programot, amivel a kő/papír/olló szerverhez kapcsolódva lehet játszani. A `rocker_client.py` fájl tartalmaz egy skeletont ehhez. A `celeryconfig.py` fájl pedig a szerver brókerének URL-jét kommentben.

Az elkészített kliens indítása:

> *python rocker\_client.py*